MyCodeSchool

Code review — alex-spring-initializare

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-23 — sha 3cb2bc8

Elev: Alex. Proiect: `cafenea-spring/`. Revizuite: `4cd1dd1` (etapa 3) și `3cb2bc8` (etapa 4, abia începută).

Rezumat față de runda 1: B1 (meniul) neatins, B2 (`@PropertySource`) parțial, B3 (etapa 3) parțial. Ai trecut la etapa 4 înainte ca verificarea etapei 3 să treacă, iar în `ProdusServiceImpl` a apărut o regresie nouă (B4). Cât timp meniul nu e cel din vanilla (B1), niciun scenariu din cerință nu se poate rula, deci nici etapele următoare nu se pot verifica.

Verificat prin rulare (pe o copie, nu în repo-ul tău):

Verificare	Rezultat
Scenariul `2,2,2,4,3,4,5,0` pe aplicația ta	`NoSuchElementException` la `ViewCafenea.java:41`
Același scenariu pe vanilla (`javac` + `java app.Main`)	meniul `=== Cafenea ===` , `Id produs: Adaugat` , bon cu `TOTAL 26.62 lei`
`--cafenea.tva=0.19`	`TOTAL 26.18 lei` ✅ (închis din runda 1)
`--cafenea.moneda=EUR`	linia de debug arată `moneda=EUR` , bonul scrie tot `TOTAL 26.62 lei`
`--spring.config.location=optional:classpath:/nu-exista.yml`	contextul nu pornește (`calculatorPret` nu găsește `${cafenea.tva}`)
`new ProdusServiceImpl().listProduse()`	`NullPointerException` (`produsRepository` e null)

🔴 Critice

B1 🔴 Meniul tot nu e cel din vanilla: scenariul din cerință se oprește cu excepție (redeschis din runda 1)

`ViewCafenea.java:49`

CUM E ACUM: `ViewCafenea` e neschimbată față de runda 1, cu excepția câmpului `CafeneaProperties` . Opțiunea 2 e tot „Comanda noua", cu bucla `do/while` care cere id-uri până la `0` (`:49–68`), iar la `4` setezi `comandaCurenta = null` (`:96`). Scenariul `2,2,2,4,3,4,5,0` nu se termină: `NoSuchElementException` la `ViewCafenea.java:41` . Pe vanilla, același input dă meniul `=== Cafenea ===` , `Id produs: Adaugat` și bonul cu `TOTAL 26.62 lei` .

DE CE: bucla de la opțiunea 2 consumă tot ce urmează din input ca id-uri de produs până întâlnește un `0` . În scenariul din cerință, `2,2,4,3,4,5` sunt luate ca id-uri (`3` și `5` dau „Nu exista acest produs", iar `0` -ul final închide bucla). Pe urmă `scanner.nextInt()` de la `:41` nu mai are ce citi, iar `Scanner` aruncă `NoSuchElementException` în loc să aștepte, pentru că stdin s-a terminat. Etapele 1–4 se verifică printr-un diff de ieșire față de vanilla tocmai ca să dovedească faptul că Spring a schimbat doar cine creează obiectele, nu ce face programul. Cât timp meniul tău e altul, nicio verificare din CERINTE (nici bonul detaliat din etapa 4) nu se poate rula așa cum e scrisă.

FIX: copiază `ViewCafenea` din `cafenea-vanilla-java/src/app/view/` și schimbă doar dependențele (vin din constructor). Opțiunea 2 = o singură cerere de id adăugată la comanda curentă, iar `4` pe o comandă goală răspunde „Comanda e goala". Apoi rulezi aceleași 8 rânduri în ambele aplicații și compari ieșirile cu `diff`.

Repro: `printf '2\n2\n2\n4\n3\n4\n5\n0\n' | java -jar target/cafenea-spring-0.0.1-SNAPSHOT.jar` -> `java.util.NoSuchElementException at mycode.products.view.ViewCafenea.play(ViewCafenea.java:41)`. Vanilla, același input: fără excepție.

Același bug la (repară-le pe toate, nu doar bucla): - `ViewCafenea.java:50` : `comandaCurenta = comandaService.comandaNoua()` la fiecare „2". Vanilla creează comanda o singură dată la start; aici fiecare „2" aruncă comanda în lucru și deschide alta (produsele adăugate anterior se pierd). - `ViewCafenea.java:59-70` : bucla `do/while` cu sentinelă `0` și promptul „ID produs: " repetat. Vanilla cere un singur id per opțiune 2. Aceasta e cauza directă a excepției. - `ViewCafenea.java:96` : `comandaCurenta = null` după finalizare. Vanilla întoarce `comandaService.comandaNoua()`, deci opțiunile 3 și 4 funcționează imediat după 4. - `ViewCafenea.java:72-76` și `:88-92` : ramurile „Nu exista o comanda activa." În vanilla comanda există mereu: `3` afișează bonul gol, iar `4` pe comandă goală scrie „Comanda e goala". - `ViewCafenea.java:88-97` : lipsește verificarea `getProduse().isEmpty()` înainte de finalizare. Vanilla scrie „Comanda e goala" și nu finalizează. - `ViewCafenea.java:41` și `:59` : `scanner.nextInt()` în loc de `Integer.parseInt(scanner.nextLine().trim())` ca în vanilla. Diferă consumul liniilor la input amestecat sau cu spații. - `ViewCafenea.java:30` : `System.out.println(cafeneaProperties)` adaugă o linie care nu există în vanilla (vezi B3). - `ViewCafenea.java:32-39` : textele meniului diferă („========== CAFENEA ==========" vs „=== Cafenea ===", „Afiseaza produse" vs „Lista produse", „Comanda noua" vs „Adauga produs in comanda", „Alege optiunea:" vs „Optiune:", „Afiseaza comenzile" vs „Comenzi finalizate"); la fel mesajele din `case 2` („Produs adaugat cu succes!" vs „Adaugat", „Nu exista acest produs" vs „Produs inexistent"). Dacă ieșirea se compară literal cu vanilla, sunt abateri. - `ViewCafenea.java:78-86` și `:100-103` : formatul bonului și al listei de comenzi (`%-15s` vs `%-12s`, spațierile de la TVA/TOTAL, `#%d %d produse %.2f` vs `#%d %d produse %6.2f`).

💡 **Hint:** pornești de la fișierul din vanilla și schimbi doar ce cere etapa 1 (dependențe prin constructor, metoda apelată din `CommandLineRunner`). Antetul cu nume și program vine abia în B3.

B2🔴 `@PropertySource` și `program.properties` lipsesc: `program` vine tot din `application.yml`

`AppConfig.java:13`

CUM E ACUM: ai scos `@PropertySource("classpath:application.yml")` (corect, nu făcea nimic), dar nu l-ai înlocuit cu nimic. `src/main/resources/` conține doar `application.yml`, unde `program` a rămas (`application.yml:13`, acum `08:00-22:00`). Importul `PropertySource` a rămas nefolosit la `AppConfig.java:9`. Cerința de etapă 3 cere `program.properties` cu `cafenea.program=08:00-20:00`, încărcat cu `@PropertySource` pe `AppConfig`.

DE CE: acum toate cheile `cafenea.*` vin dintr-o singură sursă: `application.yml`, pe care Boot îl încarcă singur. Se vede când îi iei lui Boot fișierul: cu `--spring.config.location` spre un yml inexistent, contextul nu mai pornește (`calculatorPret` nu-și găsește `${cafenea.tva}`) și nu există nicio a doua sursă care să ofere măcar `program`. Exercițiul cerea exact acea a doua sursă: un fișier pe care Boot NU îl încarcă singur, adăugat în `Environment` de `@PropertySource`. Abia acolo devine vizibilă diferența dintre „Boot citește `application.*` " și „eu adaug o sursă în plus". Fără ea, `@PropertySource` rămâne o adnotare pe care n-ai văzut-o lucrând niciodată.

FIX: `src/main/resources/program.properties` cu linia `cafenea.program=08:00-20:00`; `@PropertySource("classpath:program.properties")` pe `AppConfig`; scoate `program` din `application.yml`.

Repro: `ls src/main/resources` -> doar `application.yml`. `printf '0\n' | java -jar target/cafenea-spring-0.0.1-SNAPSHOT.jar --spring.config.location=optional:classpath:/nu-exista.yml` -> `UnsatisfiedDependencyException: Error creating bean with name 'comandaServiceImpl' ... constructor parameter 2: Error creating bean with name 'calculatorPret'`.

Același bug la (aceeași idee: valoarea nu vine de unde cere cerința): - `AppConfig.java:9` : import `PropertySource` neutilizat (nicio clasă nu are `@PropertySource`). - `AppConfig.java:28-32` : varianta comentată cu `@Value("${cafenea.nume/moneda/program}")`, care ar fi trebuit înlocuită de `program.properties`, lăsată ca cod comentat. -

`application.yml:13` : `cafenea.program` definit în yml (și `program :` cu spațiu înainte de două puncte); trebuia mutat în `program.properties` . - `CafeneaProperties.java:7` : recordul leagă `program` prin `@ConfigurationProperties(prefix = "cafenea")` doar din sursele standard Boot; fără `@PropertySource` , `program.properties` nu e încărcat (+ import `Configuration` neutilizat la `:4`). - `ViewCafenea.java:18–21` : consumă `CafeneaProperties` (`program`) fără ca valoarea să vină dintr-un `.properties` separat. - `Produs.java:28` , `ViewCafenea.java:85` și `:101` : „lei" hardcodat în loc de `cafenea.moneda` (detaliat la B3). - `AppConfig.java:23` : `@Value("${cafenea.tva}")` fără default; singura sursă e yml-ul, de aceea pică când yml-ul lipsește.

💡 **Hint:** testul tău e cel din runda 1: dacă `program` apare chiar și fără auto-încărcarea Boot a yml-ului, adnotarea lucrează.

B3 🔴 `CafeneaProperties` e injectat, dar nu e folosit: antetul și moneda nu se schimbă

`ViewCafenea.java:32`

CUM E ACUM: recordul și `@EnableConfigurationProperties` există și Spring îl injectează în `ViewCafenea` , dar singura folosire e `System.out.println(cafeneaProperties)` (`:30`), o linie de debug care afișează `CafeneaProperties[nume=test, moneda=lei, program=08:00–22:00]` . Antetul e tot „========== CAFENEA ==========" (`:32`), iar „lei" e scris de mână la `:85` și `:101` (și în `Produs.java:28`). În plus, `application.yml:10` și `:13` au acum `nume: test` și `program: 08:00–22:00` , deci verificarea `=== Cafenea · 08:00–20:00 ===` n-ar trece nici după ce schimbi antetul.

DE CE: `@ConfigurationProperties` doar umple un obiect: leagă `cafenea.nume` -> `nume` , `cafenea.moneda` -> `moneda` și așa mai departe, la pornire. Obiectul nu modifică nimic singur; codul care afișează trebuie să citească din el (`cafeneaProperties.nume()` , `.moneda()`). Proba: `--cafenea.moneda=EUR` schimbă recordul (linia de debug arată `moneda=EUR`), dar bonul arată tot `TOTAL 26.62 lei` , pentru că `printf` -ul are textul „lei" scris în cod. Lanțul yml -> `Environment` -> record funcționează; ultimul capăt, record -> ieșirea pe ecran, lipsește.

FIX: antet `System.out.printf("%n=== %s · %s ===%n", cafeneaProperties.nume(), cafeneaProperties.program());` la `:85` și `:101` folosești `cafeneaProperties.moneda()` în loc de „lei"; ștergi `println` -ul de la `:30` ; în `application.yml` pui înapoi `nume: Cafenea` ; `program` trece în `program.properties` (B2). Câmpul devine `final` .

Repro: `printf '2\n2\n4\n0\n3\n5\n0\n' | java -jar target/cafenea-spring-0.0.1-SNAPSHOT.jar --cafenea.moneda=EUR` -> prima linie `CafeneaProperties[nume=test, moneda=EUR, program=08:00–22:00]` , apoi `========== CAFENEA ==========` și `TOTAL 26.62 lei` (nu EUR).

Același bug la: - `ViewCafenea.java:32` : antetul hardcodat, în loc de `cafeneaProperties.nume()` . - `ViewCafenea.java:85` : `TOTAL` cu „lei" hardcodat, în loc de `cafeneaProperties.moneda()` . - `ViewCafenea.java:101` : lista comenzilor (`case 5`) cu „lei" hardcodat. - `Produs.java:28` : `toString` cu „lei" hardcodat (afișarea produselor, `case 1` și `2`). Modelul nu are acces la proprietăți, deci moneda trebuie formatată în view sau transmisă din afară. - `ViewCafenea.java:82–84` : `Subtotal` și `TVA` afișate fără monedă (inconsistent cu `TOTAL` ; aliniezi când repari `:85`). - `ViewCafenea.java:28` : `println(cafeneaProperties)` afișează recordul brut; `program` nu e afișat nicăieri, înlocuiește cu antetul formatat. - `AppConfig.java:28–31` : bean `CafeneaProperties` comentat (cod mort; alt subiect, doar notat la C2).

💡 **Hint:** ordinea cea mai ușoară: (1) antetul cu `nume()` și `program()` , (2) `moneda()` în cele trei locuri, (3) abia apoi ștergi linia de debug.

B4 🔴 Injectare pe câmp cu `@Autowired` : regresie față de etapa 1 (dependențele prin constructor)

`ProdusServiceImpl.java:14`

CUM E ACUM: în `4cd1dd1` ai șters constructorul și ai pus `@Autowired` pe câmpul `private ProdusRepository produsRepository` (fără `final`). Etapa 1 cere ca serviciile să-și primească dependențele prin constructor. Clasa se poate crea acum fără repository: `new ProdusServiceImpl()` compilează, iar la primul apel cade cu `NullPointerException` .

DE CE: când există un singur constructor cu parametri, Spring îl folosește singur. N-ai nevoie de `@Autowired` , iar câmpul poate fi `final` , deci obiectul nu poate exista fără dependența lui. Cu `@Autowired` pe câmp, Spring creează întâi obiectul cu constructorul implicit (câmpul e `null`) și abia apoi pune repository-ul prin reflexie. În aplicație nu se vede, pentru că Spring face ambii pași.

Oricine construiește clasa cu `new` (un test, un alt programator) primește însă un obiect pe jumătate gata, cum arată proba. În etapa 4, `@Autowired` are alt rol: pe un CONSTRUCTOR, ca să-i spună lui Spring pe care dintre doi constructori ai lui `ViewCafenea` să-l folosească.

FIX: revino la `private final ProdusRepository produsRepository;` + constructorul cu parametru, fără `@Autowired`. Folosește `@Autowired` doar în etapa 4, pe constructorul complet din `ViewCafenea`.

Repro: `Probe.java` cu `new mycode.products.services.ProdusServiceImpl().listProduse()`, compilat pe `target/classes` -> `java.lang.NullPointerException: Cannot invoke "mycode.products.repository.ProdusRepository.listProduse()" because "this.produsRepository" is null at ProdusServiceImpl.listProduse(ProdusServiceImpl.java:21)`.

Același bug la: nu există alt `@Autowired`, `@Inject` sau `@Resource` în repo (verificat cu `grep` în tot proiectul, fără `target/`). Locuri înrudite: - `ProdusServiceImpl.java:7`: import `Component` neutilizat; se rezolvă odată cu repararea clasei. - `ViewCafenea.java:18`: `private CafeneaProperties cafeneaProperties` e injectat prin constructor, dar nu e `final`, spre deosebire de celelalte dependențe din aceeași clasă (inconsecvență, nu NPE). - `ComandaServiceImpl.java:16–17` și `ViewCafenea.java:14–15`: corecte (constructor, `final`), nu sunt apariții.

🟡 Importante

M1 🟡 **`mvn test` pică** (din runda 1, neatins). `CafeneaSpringApplicationTests` (pachetul `mycode`): `Unable to find a @SpringBootConfiguration by searching packages upwards`. Clasa principală stă în `mycode.pornire`, adică sub test, nu deasupra lui.

M2 🟡 **`@Import(AppConfig.class)` de prisos** (din runda 1, neatins). `CafeneaSpringApplication.java:11`: `AppConfig` stă în `mycode.config`, deci în interiorul `@ComponentScan("mycode")`.

M3 🟡 **`CalculatorPret` nu e `@Component`.** `AppConfig.java:22–25`: TVA-ul vine acum din yml (`--cafenea.tva=0.19` -> `TOTAL 26.18` ✅), dar prin `@Value` pe parametrul unui `@Bean`. Cerința: `@Component` pe `CalculatorPret` + `@Value` în constructorul lui, iar în `AppConfig` rămâne un singur `@Bean` (`Scanner`). Acum sunt 3.

M4 🟡 **Etapa 4 abia începută** (`3cb2bc8`). `BonPrinter`, `DetaliatBonPrinter` și `Reducere` sunt clase goale; `BonPrinter` și `Reducere` trebuie să fie interfețe; lipsesc `SimpluBonPrinter`, `ReducereMeniu` și `ReducereFidelitate`; pachetele cerute sunt `comenzi/bon` și `comenzi/reduceri` (la tine `orders/printer` și `orders/service`). Termină B1–B3 înainte, altfel nu poți verifica etapa 4 pe scenariul din cerință.

🟢 Cleanups

- **C1** (din runda 1, extins) importuri nefolosite: `Component` în `ProdusRepositoryImpl.java:4`, `ComandaRepositoryImpl.java:4`, `ProdusServiceImpl.java:6`, `ComandaServiceImpl.java:7`; `PropertySource` în `AppConfig.java:9`; `Configuration` în `CafeneaProperties.java:4`.
- **C2** cod comentat rămas în `AppConfig.java:27–32` (vechiul `@Bean cafeneaProperties`); șterge-l, e în istoricul git.
- **C3** (din runda 1) `application.yml:13` `program :` are spațiu înainte de `:`; oricum se mută în `program.properties`.
- **C4** (din runda 1) `ViewCafenea` în `products.view`, deși lucrează și cu comenzile; un pachet `view` separat ar reflecta mai bine rolul ei.
- **C5** formatare: `private CafeneaProperties` (2 spații, fără `final`) la `ViewCafenea.java:18`, `String nume ,String moneda` în `CafeneaProperties.java:7`, `@Value("${cafenea.tva}")double` fără spațiu.

Before / After

Înainte	După (indicativ, în document, nu în codul tău)
`ViewCafenea` : `case 2:` cu `comandaCurenta = comandaService.comandaNoua()` și buclă `do/while` până la `0`	`case 2 ->` o singură cerere de id, adăugată la comanda curentă (ca în vanilla); `case 4` pe comandă goală: „Comanda e goala”; după finalizare `comandaCurenta = comandaService.comandaNoua()`
`application.yml` cu `program: 08:00-22:00` ; fără `program.properties` ; `AppConfig` fără `@PropertySource`	`src/main/resources/program.properties` cu `cafenea.program=08:00-20:00` ; `@PropertySource("classpath:program.properties")` pe `AppConfig` ; `program` scos din yml
`System.out.println(cafeneaProperties);` + `"========== CAFENEA =========="` + `"... lei"`	`System.out.printf("%n=== %s · %s ===%n", cafeneaProperties.nume(), cafeneaProperties.program());` + `cafeneaProperties.moneda()` în loc de „lei”
`@Autowired` `private ProdusRepository produsRepository;`	`private final ProdusRepository produsRepository;` `public ProdusServiceImpl(ProdusRepository produsRepository) { this.produsRepository = produsRepository; }`

Q&A

1. Cu `--cafenea.moneda=EUR` , prima linie afișează `moneda=EUR` , dar bonul scrie tot „lei”. Ce a făcut Spring cu valoarea `EUR` și ce pas lipsește ca să ajungă în bon?
2. Dacă muți `program` în `program.properties` și scoți `@PropertySource` , ce valoare va avea `cafeneaProperties.program()` ? Dar `@Value("${cafenea.program}")` ? De ce se comportă diferit cele două?
3. În `ProdusServiceImpl` , ce se întâmplă, pas cu pas, între momentul în care Spring creează obiectul și momentul în care câmpul cu `@Autowired` primește valoare? De ce nu poate exista acest moment intermediar când dependența vine prin constructor?

Runda 1 — 2026-10-22 — sha 57e0657

- 🔴 B1 meniul ≠ vanilla: NEATINS. Același `NoSuchElementException` pe `2,2,2,4,3,4,5,0` (acum la `ViewCafenea.java:41`), vezi B1 runda 2
- 🟡 B2 `@PropertySource` pe yml: PARȚIAL. Adnotarea greșită e scoasă, dar `program.properties` și `@PropertySource` lipsesc, vezi B2 runda 2
- 🟡 B3 etapa 3 pe jumătate: PARȚIAL. ✅ TVA din yml (`--cafenea.tva=0.19` -> `TOTAL 26.18` , verificat prin rulare); ✅ record `CafeneaProperties` + `@ConfigurationProperties` + `@EnableConfigurationProperties` ; 🔴 antetul și moneda neaplicate (B3 runda 2); 🟡 `CalculatorPret` nu e `@Component` , `AppConfig` are tot 3 `@Bean` (M3 runda 2)
- 🟡 M1 `mvn test` : NEATINS (verificat, aceeași eroare) · M2 `@Import` : NEATINS
- 🟢 C1 importuri: NEATINS (+2 noi) · C2 `program :` : NEATINS (C3 runda 2) · C3 pachet view: NEATINS (C4 runda 2)